

Trabajo Práctico Integrador – Programación II

Food Store – Sistema de Gestión de Pedidos de Comida

Alumno - Grupo n° 31

Rolando Nebreda – DNI 18.350.686

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Programación II

Coordinador

Carlos Martínez

Docente Titular

Cinthia Rigoni

Docente Tutor

Sofía Raia

13 de Julio de 2026

Tabla de contenido

Trabajo Práctico Integrador	3
Introducción	3
Objetivos de Aprendizaje	3
Alcance de programación	4
Modelo UML y Entidades	4
Consigna	5
Marco Teórico	7
Programación Orientada a Objetos	7
UML y Relaciones	7
Colecciones en Java	8
Interfaces y Excepciones	8
Genéricos	8
Desarrollo	9
Decisiones Técnicas y Arquitectura	9
Menú Principal	10
1 - Gestión de Categorías	10
2 - Gestión de Productos	11
3 - Gestión de Usuarios	11
4 - Gestión de Pedidos y Detalles	12
Dificultades y soluciones	14
Reglas de negocio y funcionalidades adicionales	14
Elaboración del Sistema	16
Referencias	19
Links al Repositorio y Video	19
Repositorio GitHub	19
Video explicativo	19

Trabajo Práctico Integrador

Introducción

Se propone desarrollar un sistema de consola llamado Food Store, orientado a la gestión de productos de un negocio de comidas mediante una aplicación Java basada en Programación Orientada a Objetos (POO).

El sistema permitirá gestionar categorías, productos, usuarios y pedidos (según el UML provisto), realizando operaciones CRUD completas desde un menú de opciones en consola. El acceso se realizará directamente a través del menú para ejecutar altas, bajas, modificaciones y consultas (no se implementará login).

Como el sistema no contará con base de datos, toda la información se almacenará en memoria durante la ejecución del programa mediante el uso de Colecciones.

El desarrollo deberá realizarse tomando como guía un backlog de épicas e historias de usuario. Esto implica transformar cada historia en opciones concretas del menú. Además, se deberán incorporar validaciones de entrada, manejo de excepciones propias y una organización clara de responsabilidades (modelo, servicios/lógica y vista por consola).

Objetivos de Aprendizaje

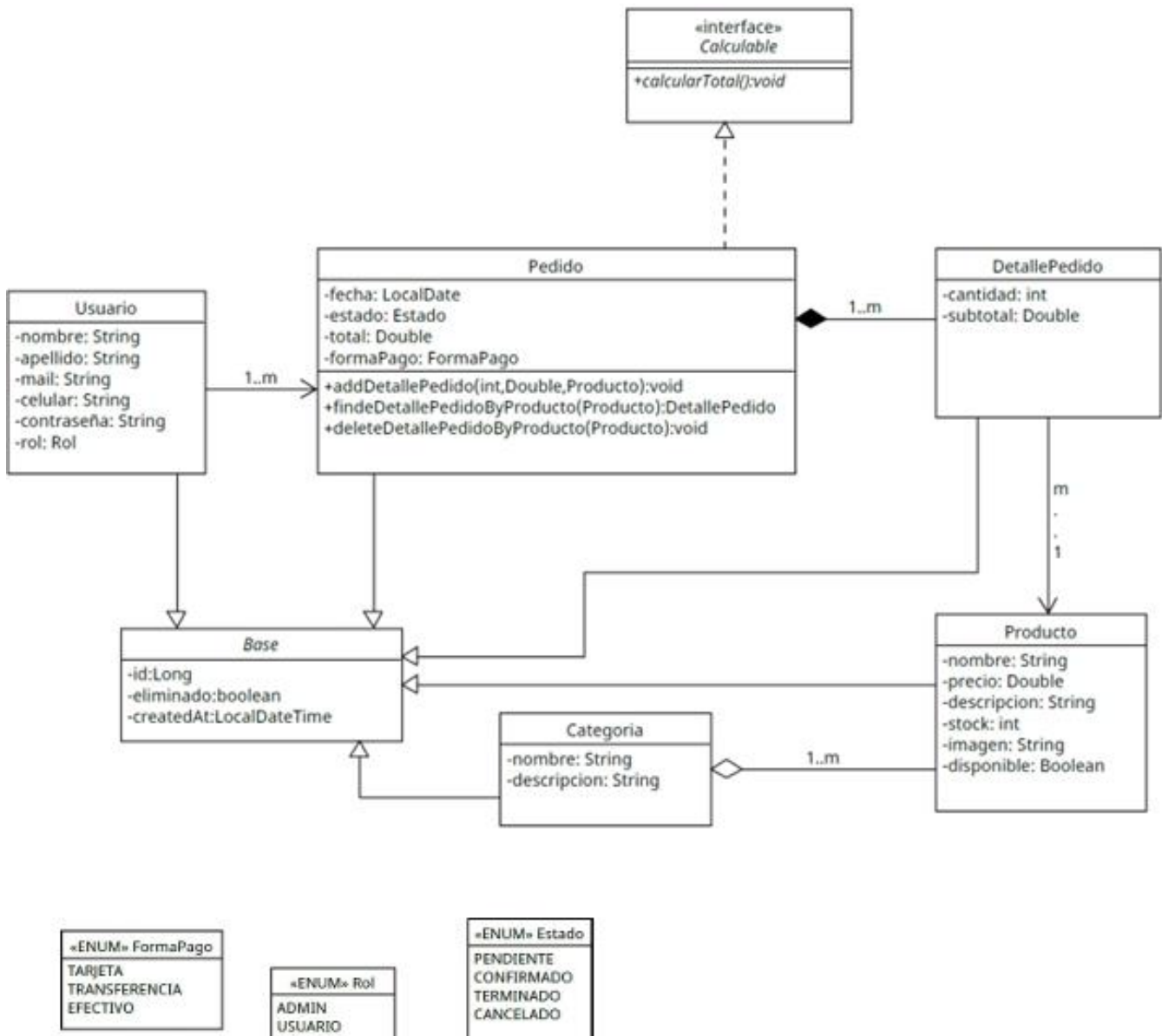
Al finalizar el TPI, el proyecto debe evidenciar que el estudiante puede:

- Modelar correctamente clases y relaciones del UML (asociaciones 1..N y N..1).
- Aplicar herencia (clase base) y sobrescritura de métodos (toString, equals).
- Utilizar estructuras de datos dinámicas (Colecciones) para gestionar las entidades.
- Implementar Interfaces para estandarizar comportamientos (ej. Calculable).
- Gestionar errores creando y capturando Excepciones propias (por ejemplo, EntidadNoEncontradaException, StockInvalidoException).

Alcance de programación

- Toda la interacción ocurre en consola mediante un menú.
- Aplicar POO (encapsulamiento, constructores, herencia, polimorfismo) según UML.
- Utilizar Colecciones para el almacenamiento y gestión de los objetos en memoria.
- Implementar Interfaces para definir comportamientos comunes.
- Aplicar manejo de Excepciones para validar errores y evitar cierres inesperados del programa.

Modelo UML y Entidades



Consigna

El sistema se basa en un diseño orientado a objetos donde varias clases comparten atributos comunes mediante herencia, implementando interfaces para comportamientos específicos:

- **Clase Abstracta Base:** id (Long), eliminado (boolean), createdAt (LocalDateTime).

Todas las entidades a continuación heredan de esta clase.

- **Categoria:** hereda de Base + nombre, descripcion (y relación 1:N con Producto).

- **Producto:** hereda de Base + nombre, precio, descripcion, stock, imagen, disponible (y relación N:1 con Categoria).

- **Usuario:** hereda de Base + nombre, apellido, mail (único), celular, contraseña, rol (y relación 1:N con Pedido).

- **Pedido:** hereda de Base + fecha (LocalDate), estado, total, formaPago (y relación de composición 1:N con DetallePedido).

- Implementa la interfaz Calculable (método calcularTotal()).

- Métodos propios:

addDetallePedido(int, Double, Producto),

findeDetallePedidoByProducto(Producto),

deleteDetallePedidoByProducto(Producto).

- **DetallePedido:** hereda de Base + cantidad, subtotal (y relación N:1 con Producto).

- **Enums:** **Rol** (ADMIN, USUARIO), **Estado** (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO), **FormaPago** (TARJETA, TRANSFERENCIA, EFECTIVO).

- **Interfaz: Calculable** (define el comportamiento para calcular totales).

Interfaz de Consola (Menú):

El sistema debe iniciar mostrando un menú principal. Cada opción abre un submenú CRUD.

Pautas importantes del menú

- El menú debe validar entradas (opciones fuera de rango, ids inexistentes, números mal ingresados).

- Las operaciones deben mostrar mensajes claros (éxito / error) y retornar al menú.

- Para facilitar pruebas, es válido listar elementos antes de pedir un ID (por ejemplo: listar categorías y luego pedir id).

Regla general

- Main/Menu: solo interacción con usuario (Scanner) y armado de objetos.
- Service: coordinación de casos de uso (validaciones, reglas de negocio simples).
- Entities: modelo de dominio (POO), con toString() útil para listados en consola.

Reglas mínimas de negocio

Además del CRUD, se espera que el sistema respete reglas básicas para que el dominio tenga sentido:

- No permitir crear Producto con precio < 0 o stock < 0 (Lanzar excepción).
- No permitir crear Pedido sin usuario.
- Al crear DetallePedido, la cantidad debe ser > 0.
- Mail de Usuario debe ser único (validar recorriendo la colección e informar error).
- Soft delete: marcar eliminado = true en vez de remover físicamente el objeto de la colección, para mantener el historial.

Notas

- No hay login: se asume un usuario operador que accede directamente a los menús de gestión.
- Todas las bajas son lógicas (soft delete) usando el campo eliminado = true.
- Las interacciones se realizan mediante menús de consola.

Marco Teórico

Programación Orientada a Objetos

La POO organiza el software en **clases y objetos**.

- **Clase:** plantilla que define atributos y métodos.
- **Objeto:** instancia concreta de una clase.
- **Encapsulamiento:** oculta detalles internos y expone acceso controlado mediante modificadores (public, private, protected) y **getters/setters**.
- **Herencia:** permite crear subclases a partir de clases existentes, reutilizando código.
- **Polimorfismo:** un mismo método se comporta distinto según el objeto que lo invoque.
- **Abstracción:** se logra con **clases abstractas y métodos abstractos**, que definen contratos sin implementación.
- **Interfaces:** contratos que garantizan comportamientos comunes en distintas clases.

Palabras clave:

this: referencia al objeto actual.

super: referencia a la superclase inmediata, usada para invocar constructores o métodos del padre.

final: restringe cambios (variables constantes, métodos no sobrescribibles, clases no heredables).

UML y Relaciones

El **UML (Unified Modeling Language)** es un lenguaje gráfico para representar sistemas.

Relaciones 1 a 1 y 1 a N: pueden ser **unidireccionales** o **bidireccionales**, según si ambas clases se conocen mutuamente.

- **Asociación:** referencia simple entre clases.
- **Agregación:** relación flexible, las partes pueden existir por separado.

- **Composición:** relación fuerte, el ciclo de vida de las partes depende del todo.

Dependencias: usos temporales entre clases (por parámetros o creación de objetos).

Colecciones en Java

Estructuras para manejar grupos de datos:

List: ordenada, permite duplicados (***ArrayList***). (Solo las utilizadas en el TPI)

Iterator: recorre colecciones de forma uniforme con **hasNext ()** y **next ()**.

For-Each: sintaxis simplificada para recorrer colecciones.

Enums: valores estandarizados para evitar errores de entrada.

Comparator: interfaz para definir criterios de ordenamiento personalizados, separando la lógica de comparación de la clase misma.

Interfaces y Excepciones

Interfaces: contratos que definen métodos abstractos, constantes y, desde Java 8, métodos ***default*** y ***static***.

Excepciones: eventos anómalos que interrumpen el flujo normal. Se manejan con ***try-catch-finally***, ***multicatch*** y ***try-with-resources***.

Excepciones personalizadas: permiten representar errores específicos del dominio.

Checked vs Unchecked: las primeras deben declararse o capturarse, las segundas no.

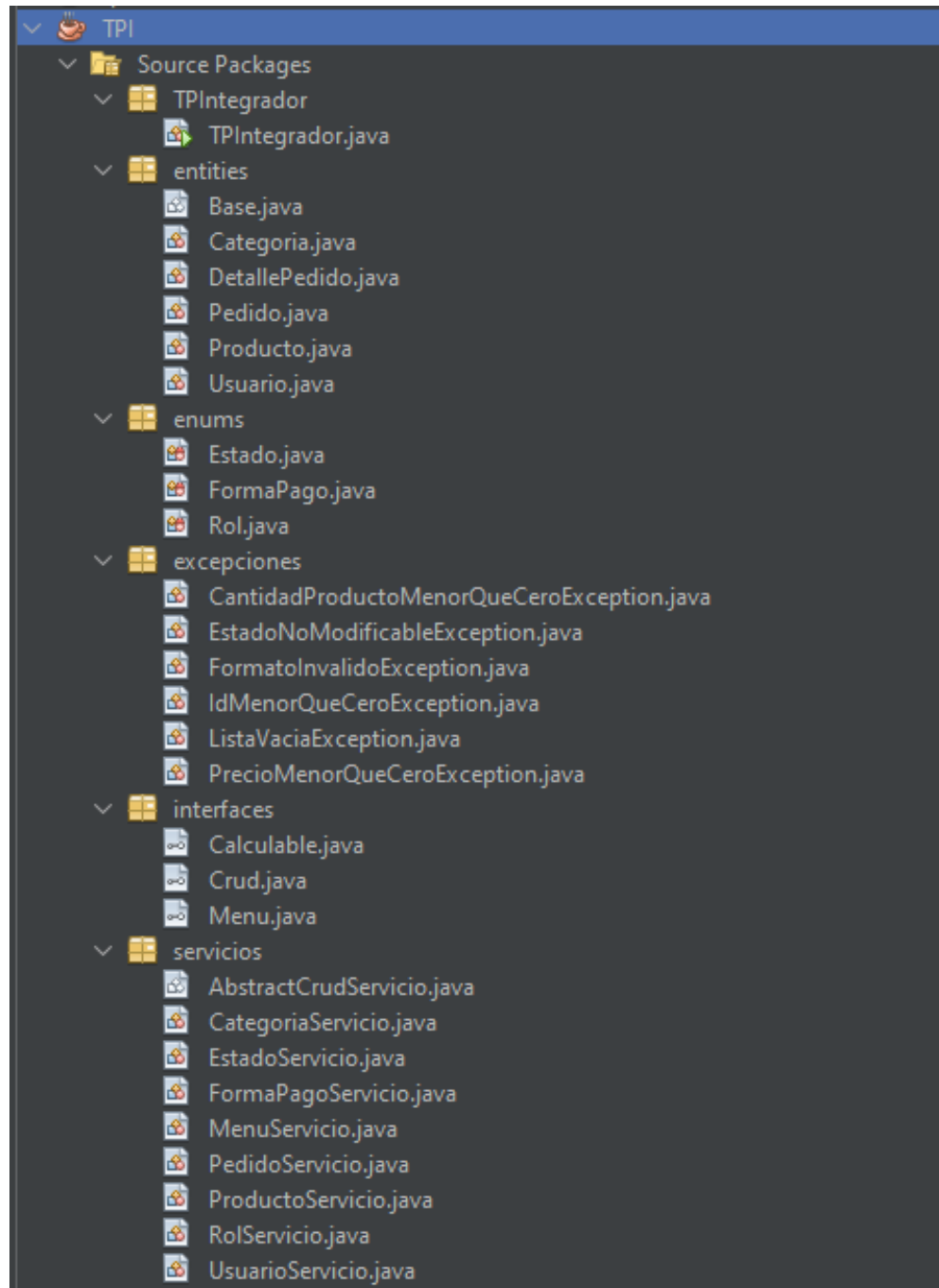
Genéricos

Los **genéricos** permiten escribir código parametrizado en tipos (***<T>***), logrando **polimorfismo paramétrico**.

Desarrollo

Decisiones Técnicas y Arquitectura

Organización de paquetes



Menú Principal

```
=====
SISTEMA DE PEDIDOS (FOOD STORE)
=====
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
=====
Elija una opción:
```

Al ingresar al Sistema se ofrece un menú de opciones para acceder a los distintos módulos del programa:

1 - Gestión de Categorías

```
=====
CATEGORIAS
=====
1. Agregar
2. Buscar por Id o Nombre
3. Modificar
4. Eliminar
5. Listar
0. Salir
=====
Elija una opción: |
```

Editar categoría

```
=====
MODIFICAR CATEGORIA
=====
1. Nombre
2. Descripción
0. Salir
=====
Elija una opción: |
```

2 - Gestión de Productos

```
=====
PRODUCTOS
=====
1. Agregar
2. Buscar por Id o Nombre
3. Modificar
4. Eliminar
5. Listar
6. Reporte de Productos sin stock
7. Reporte de Productos con Categoría Eliminada
0. Salir
=====
Elija una opción:
```

Editar producto

```
=====
MODIFICAR PRODUCTO
=====
1. Nombre
2. Precio
3. Descripción
4. Stock
5. Imagen
6. Disponible
7. Categoría
0. Salir
=====
Elija una opción: |
```

3 - Gestión de Usuarios

```
=====
USUARIOS
=====
1. Agregar
2. Buscar por Id
3. Modificar
4. Eliminar
5. Listar
0. Salir
=====
Elija una opción:
```

- El Ingreso de Rol (y modificación) se realiza accediendo a un menú.

```
=====
ROLES
=====
1. ADMIN
2. USUARIO
0. Salir
=====
Elija una opción:
```

Editar usuario

```
=====
MODIFICAR USUARIO
=====
1. Nombre
2. Apellido
3. Rol
4. Mail
5. Celular
0. Salir
=====
Elija una opción: |
```

4 - Gestión de Pedidos y Detalles

```
=====
PEDIDOS
=====
1. Pedido nuevo
2. Buscar Pedido por Id
3. Modificar Datos Pedido
4. Eliminar
5. Listar
6. Detalles
0. Salir
=====
Elija una opción:
```

- El Ingreso de Forma de Pago (y modificación) se realiza accediendo a un menú.

```
=====
FORMA DE PAGO
=====
1. TARJETA (TC)
2. TRANSFERENCIA (TR)
3. EFECTIVO (EF)
0. Salir
=====
Elija una opción:
```

Actualizar estado/forma de pago del pedido

```
=====
MODIFICAR DATOS PEDIDO
=====
1. Forma de Pago
2. Estado
0. Salir
=====
Elija una opción:
```

- La modificación del estado y forma de pago se realiza accediendo a un menú.

```
=====
ESTADO DE PEDIDO
=====
1. PENDIENTE
2. CONFIRMADO
3. TERMINADO
4. CANCELADO
0. Salir
=====
Elija una opción:
```

Menú Detalles (opción 6 de Gestión de pedidos)

```
=====
DETALLES DE PEDIDO
=====
1. Agregar Detalle a pedido
2. Buscar Detalle por Producto
3. Modificar Detalle por Producto
4. Eliminar Detalle por producto
0. Salir
=====
Elija una opción: |
```

Solo se puede acceder si el pedido está en estado **Pendiente**.

Dificultades y soluciones

Reglas de negocio y funcionalidades adicionales

¿Qué sucede cuando se elimina una Categoría con Productos asociados?

Se puede eliminar. Primero se eliminan los productos de la lista productos de la Categoría, previa modificación del atributo Categoría del producto.

Todos los productos asociados pasarán a tener su atributo Categoría = null, pero en reportes se verá como “**sin categorías**”. El atributo **disponible** de dichos productos, pasará a **false** a pesar de tener stock, y no podrán ser utilizados en nuevos pedidos o modificar detalles que contengan ese producto hasta que se le asocie una nueva Categoría.

¿Se puede Listar productos por Categoría?

La respuesta es Sí. Para acceder a dicho reporte se ingresa al menú 1. Categorías \ 2. Buscar por Id o nombre. En este punto del menú se elige la categoría y muestra, además de id, nombre y descripción de la categoría seleccionada, la lista de productos asociados a la misma.

Menús de gestión y modificación.

Todos los elementos (Categorías, Productos, Usuarios, Pedidos) se gestionan accediendo al menú CRUD correspondiente. Lo mismo para el ingreso y modificación de Rol, Forma de Pago y Estado. Además las modificaciones de los atributos, también se realizan con un menú de modificación asociado a cada elemento.

¿Puede eliminarse un Producto referenciado en uno o varios Pedidos?

Sí. Puede ser eliminado, y no estará disponible para nuevos pedidos o modificar atributos de ese producto.

Reporte de productos sin stock

Muestra lista de productos con stock = 0, a fin de reponer stocks. Estos productos no podrán ser utilizados en nuevos pedidos en esa condición (**disponible = false**).

Reporte de productos con categoría eliminada

Muestra lista de productos con categoría eliminada (“sin categoría”) a fin de asociarlos a una nueva categoría. Estos productos no podrán ser utilizados en nuevos pedidos ni modificados en esa condición (disponible = false) aunque cuenten con stock.

Validación de número de celular

Se realiza validación del formato del número de celular. Debe ser numérico (aunque el dato es del tipo String de acuerdo a la consigna), mayor o igual a cero y con un máximo de quince cifras:

```
formatoValido = nroCelular >= 0L && nroCelular < Math.pow(10, 15);
```

Si el valor ingresado es igual a cero, toma el valor “no posee”.

Validación de mail

Se realiza validación del formato del número. Debe ser texto (no nulo, ni vacío).

Verifica que contenga una “@”, que sea única y que no esté ni al principio ni al final del mail. Si el valor ingresado es igual a “0”, toma el valor “no posee”.

```
stringCorrecto = string != null && !string.isEmpty() && !string.isBlank();
```

```
formatoValido = stringCorrecto(mail)
```

```
&& (((mail.length() - mail.replace("@", "").length()) == 1 &&
```

```
!mail.startsWith("@") && !mail.endsWith("@")) || mail.equals("0"));
```

El mail debe ser único: si ya existe, se informa el error y se solicita otro.

¿Se puede Listar pedidos por Usuario?

Sí. Para acceder a dicho reporte se ingresa al menú 3. Usuarios \ 2. Buscar por Id. En este punto del menú se elige el usuario por id, y muestra, además de los datos del usuario seleccionado, la lista de pedidos asociados al mismo con subtotal por pedido y total.

¿Qué sucede cuando se elimina un Usuario con sus Pedidos asociados?

Previo a eliminar el usuario, los pedidos en estado Pendiente, se borran. Pedidos en estado Confirmado, pasan a Terminado. Si hay Terminados o Cancelados quedan igual.

Estados de un Pedido

Pendiente: El pedido se crea con este estado. Se asume que es un pedido en elaboración y que por lo tanto puede ser modificado al igual que sus detalles. En este estado además, el pedido puede existir sin detalles, solo con datos de usuario y forma de pago. En tal caso si el pedido en estado Pendiente aún no posee detalles, no puede cambiar a otro estado.

Confirmado: Una vez en este estado ya no puede volver al anterior. Se asume que se definió la composición del pedido y está listo para preparar y entregar.

Terminado: una vez en este estado el pedido no vuelve a los estados anteriores. Se asume que el pedido fue entregado correctamente.

Cancelado: una vez en este estado el pedido no vuelve a los estados anteriores. Se asume que algo salió mal. El pedido no fue entregado correctamente y además los productos no volverán al stock.

¿Pueden cargarse dos Detalles con el mismo Producto en el mismo Pedido?

No. Se debe modificar el Detalle existente y agregar la cantidad deseada.

¿Puede ser eliminado un Pedido que posee uno o varios Detalles?

El pedido puede ser eliminado “**sólo**” en estado Pendiente. Si existen detalles estos también se eliminarán, y los productos asociados “devolverán” la cantidad actual del detalle al stock del producto correspondiente.

Modificar Detalles de Pedido

Al modificar un detalle de pedido, se puede cambiar solamente la cantidad del producto. Se considera en la verificación del máximo a cargar, el stock del producto más la cantidad actual que existe en el detalle.

Elaboración del Sistema

Además de las entidades con sus atributos y métodos, relaciones y funcionalidad de acuerdo al Modelo UML y a la Consigna de este trabajo (páginas 5 y 6 de este documento), el programa se elaboró de la siguiente manera:

Main

TPIntegrador (main)

La clase main TPIntegrador solo accede a la clase de servicio MenuServicio y genera una instancia de la misma para utilizar el método mostrarMenu()

Interfaces

Interface genérica Crud

Ofrece un “contrato” de métodos “genéricos” a ser implementados por las clases que necesitan desarrollar un CRUD. (El tipo de dato <T> se referencia en la clase que la implementa).

Interface Menu

Posee métodos a ser implementados en clases que utilicen un menú de opciones. Estos métodos son default.

Clases de servicio

AbstractCrud Servicio

```
public abstract class AbstractCrudServicio<T extends Base> implements Crud<T>{
```

Clase abstracta de servicio con un CRUD genérico, a implementar por las clases hijas. Las clases que la heredan son clases de servicios de entidades heredadas de Base (Como Categoria, Producto, etc.). Implementa la interface genérica Crud.

El atributo elementos, es una lista genérica que almacenará los objetos de cada tipo creados en el programa. Por ejemplo la clase de servicio CategoriaServicio en su Lista **elementos** almacenará Categorías. Mientras que la clase de servicio ProductoServicio en su lista **elementos** guardará Productos.

MenuServicio

```
public class MenuServicio implements Menu{
```

Menú Principal. Implementa Interface Menu

CategoriaServicio

```
public class CategoriaServicio extends AbstractCrudServicio<Categoria> implements Menu{
```

Servicios para la clase Categoría (collection + metodos crud + menu)

ProductoServicio

```
public class ProductoServicio extends AbstractCrudServicio<Producto> implements Menu{
```

Servicios para la clase Producto (collection + metodos crud + menu)

UsuarioServicio

```
public class UsuarioServicio extends AbstractCrudServicio<Usuario> implements Menu{
```

Servicios para la clase Usuario (collection + metodos crud + menu)

PedidoServicio

```
public class PedidoServicio extends AbstractCrudServicio<Pedido> implements Menu{
```

Servicios para la clase Pedido y DetallePedido (collection + metodos crud + menu)

EstadoServicio

```
public class EstadoServicio implements Menu{  
Servicios para la clase enum Estado (menu)
```

FormaPagoServicio

```
public class FormaPagoServicio implements Menu{  
Servicios para la clase enum FormaPago (menu)
```

RolServicio

```
public class RolServicio implements Menu{  
Servicios para la clase enum Rol (menu)
```

Excepciones

CantidadProductoMenorQueCeroException: Cantidad ingresada inválida

EstadoNoModificableException: se arroja cuando se intenta modificar un pedido que se encuentra en un estado en el cual no puede ser modificado

FormatoInvalidoException: error de tipo de dato o formato

IdMenorQueCeroException: se arroja con la condición $id < 0$

ListaVacíaException: se arroja cuando se intenta listar u operar sobre una lista que no tiene elementos.

PrecioMenorQueCeroException: precio ingresado inválido.

Referencias

- Documentación oficial de Java:
<https://docs.oracle.com/javase/>
- Tutoriales de Java I/O (Input/Output): (docs.oracle.com in Bing)
<https://docs.oracle.com/javase/tutorial/essential/io/>
- GeeksforGeeks – Java Collections: (geeksforgeeks.org in Bing)
<https://www.geeksforgeeks.org/collections-in-java/>
- Charly Cimino programación:
<https://charlycimino.com/>

Tecnicatura Universitaria en Programación. Programación II. Apuntes de la materia:

Unidad 4: Programación Orientada a Objetos

Unidad 5: UML (Unified Modeling Language)

Unidad 6: Colecciones

Unidad 7: Herencia y Polimorfismo

Unidad 8: Interfaces y Excepciones

Unidad 9: Genéricos

Links al Repositorio y Video

Repositorio GitHub

<https://github.com/rnebreda/TP-INTEGRADOR---PROGRAMACION-II>

Video explicativo

<https://youtu.be/C1jrEngVuNs>